

TASTY RECIPES

SOFTWARE REQUIREMENTS SPECIFICATION

Prepared by
Muhammad Awais Khalid

Version 1.0
September 2026

Professional requirements, design, testing, deployment, and user-interface documentation.

DOCUMENT CONTROL

Item	Details
Document Title	Tasty Recipes - Software Requirements Specification
Version	1.0
Prepared By	Muhammad Awais Khalid
Project Type	Responsive static front-end web application
Technology	HTML5, CSS3, Vanilla JavaScript
Version Control	Git / GitHub
Deployment	GitHub Pages
Document Status	Release-ready project documentation

Revision History

Version	Date	Summary
0.1	September 2026	Initial HTML-focused specification.
1.0	September 2026	Updated to implemented responsive UI, JavaScript interactions, nine recipe detail pages, dropdown navigation, search/filtering, validation, image optimization, testing, and UI evidence placeholders.

TABLE OF CONTENTS

Chapter 1: Introduction	8
1.1 Introduction	8
1.2 Project Background	8
1.3 Problem Definition	8
1.4 Project Objectives	8
1.5 Project Scope	8
1.6 User Needs	8
1.7 Constraints and Assumptions	9
Chapter 2: Introduction to Proposed System	10
2.1 Introduction	10
2.2 Proposed Solution	10
2.3 Target Users	10
2.4 Product Features	10
2.5 Operating Environment	10
2.6 System Context	11
Chapter 3: Requirement Specifications	12
3.1 Introduction	12
3.2 Functional Requirements	12
3.3 Fully Dressed Use Cases	12
3.4 External Interface Requirements	13
3.5 Non-Functional Requirements	13
3.6 Acceptance Criteria	14
Chapter 4: Design Specifications	15
4.1 Introduction	15
4.2 System Architecture	15
4.3 Website Structure	15
4.4 Use Case Diagram	16
4.5 Block Diagram	16
4.6 Class Diagram	16
4.7 Sequence Diagram	17
4.8 File and Folder Design	17
4.9 User Interface Design	17
Chapter 5: Test Specifications	23
5.1 Introduction	23
5.2 Test Strategy	23
5.3 Test Cases	23
5.4 Requirements Traceability Matrix	23
5.5 Final QA Checklist	24
Chapter 6: Deployment and Conclusion	25
6.1 Deployment Requirements	25
6.2 Deployment Procedure	25
6.3 Current Limitations	25

6.4 Future Enhancements	25
6.5 Conclusion	25
Appendix A: Glossary	26
Appendix B: User Manual	27
Bibliography	28

LIST OF TABLES

Table 1.1: Project Objectives8

Table 2.1: Target Users10

Table 2.2: Product Features 10

Table 3.1: Functional Requirements12

Table 3.2: UC-01 Browse and Filter Recipes 12

Table 3.3: UC-02 View Recipe Details 13

Table 3.4: UC-03 View About Information13

Table 3.5: UC-04 Use Contact Form 13

Table 3.6: External Interface Requirements 13

Table 3.7: Non-Functional Requirements13

Table 5.1: Test Cases.....23

Table 5.2: Requirements Traceability Matrix23

Table B.1: Main User Actions27

LIST OF FIGURES

Figure 2.1: System Context Diagram.....	11
Figure 4.1: Website Navigation / Site Map	15
Figure 4.2: Use Case Diagram	16
Figure 4.3: High-Level Block Diagram	16
Figure 4.4: Conceptual Class Diagram	16
Figure 4.5: Sequence Diagram - View Recipe Details	17
Figure 4.6: Home Page UI	18
Figure 4.7: Recipe Catalogue UI	19
Figure 4.8: Recipe Detail UI	20
Figure 4.9: About Page UI	21
Figure 4.10: Contact Page UI	21
Figure 4.11: Mobile / Responsive UI	22

LIST OF ABBREVIATIONS

Abbreviation	Term
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
UI	User Interface
UX	User Experience
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
HTTPS	Hypertext Transfer Protocol Secure
UC	Use Case
TC	Test Case
RTM	Requirements Traceability Matrix
DOM	Document Object Model
URL	Uniform Resource Locator

Chapter 1: INTRODUCTION

1.1 Introduction

Tasty Recipes is a responsive multi-page recipe website developed as a portfolio-quality front-end project. The implemented release uses semantic HTML5 for structure, CSS3 for responsive presentation and animation, and Vanilla JavaScript for navigation behavior, recipe search/filtering, scroll-reveal effects, header behavior, and client-side contact-form validation.

1.2 Project Background

The project demonstrates a complete small-software workflow: requirements definition, interface design, implementation, testing, Git-based version control, documentation, image optimization, and static deployment. The recipe domain provides realistic content while keeping the technical scope focused on front-end engineering.

1.3 Problem Definition

Simple learning websites often lack consistent navigation, responsive behavior, documented requirements, reusable design rules, testing evidence, and release discipline. Tasty Recipes addresses these issues by implementing a coherent multi-page interface with documented requirements and traceable tests.

1.4 Project Objectives

ID	Objective
OBJ-01	Build a professional responsive multi-page recipe website.
OBJ-02	Provide a searchable and category-filterable recipe catalogue.
OBJ-03	Provide nine complete recipe detail pages with ingredients and ordered instructions.
OBJ-04	Maintain consistent global navigation, including a Recipes dropdown.
OBJ-05	Provide client-side contact-form validation without claiming backend delivery.
OBJ-06	Optimize high-resolution food imagery for visual quality and web performance.
OBJ-07	Document, test, version, and deploy the project using professional practices.

Table 1.1: Project Objectives

1.5 Project Scope

Version 1.0 includes Home, Recipes, About, Contact, and nine individual recipe detail pages. It includes responsive styling, navigation dropdowns, category filtering through URL parameters, catalogue search, image-based recipe cards, ingredient checklists, step-by-step instructions, responsive layouts, animations, and front-end form validation.

The project does not include a backend, database, authentication, user accounts, persistent favorites, comments, admin management, or real contact-message delivery.

1.6 User Needs

- Fast and predictable navigation.
- Clear recipe categories and search.
- Readable ingredients and instructions.
- Sharp, relevant food images.
- Responsive behavior on desktop, tablet, and mobile.
- Clear feedback when contact-form input is invalid.

1.7 Constraints and Assumptions

- The application remains a static front-end website.
- JavaScript is executed in a modern browser.
- GitHub Pages or equivalent static hosting is sufficient.
- Recipe content is maintained manually in HTML files.
- Contact-form validation is client-side only; no message is transmitted.
- High-resolution source images are optimized before release.

Chapter 2: INTRODUCTION TO PROPOSED SYSTEM

2.1 Introduction

The proposed and implemented system is a static front-end recipe website that allows visitors to discover recipes, filter the catalogue by category, search by recipe name, open complete recipe pages, read project information, and interact with a validated contact-form interface.

2.2 Proposed Solution

A shared visual design system is implemented through `assets/css/style.css`, while `assets/js/main.js` centralizes client-side behavior. Relative links connect root pages and recipe-detail pages. The recipe catalogue accepts category query parameters such as `?category=dessert` so navigation dropdown items can open a pre-filtered catalogue.

2.3 Target Users

User	Description
Visitor	Browses, searches, filters, and reads recipes.
Portfolio Reviewer	Evaluates front-end structure, design quality, responsiveness, and engineering practices.
Student Developer	Studies the relationship between requirements, implementation, testing, Git workflow, and deployment.

Table 2.1: Target Users

2.4 Product Features

Feature ID	Feature Name	Description
FID01	Professional Home Page	Hero, featured recipes, statistics, value proposition, and CTA.
FID02	Recipe Catalogue	Nine recipe cards with category metadata.
FID03	Search	Client-side recipe-name search.
FID04	Category Filtering	All, Breakfast, Lunch, Dinner, Healthy, Dessert.
FID05	Recipe Dropdown	Consistent dropdown navigation on all pages.
FID06	Recipe Detail Pages	Nine dedicated pages with images, metadata, ingredients, steps, and tips.
FID07	Responsive UI	Desktop, tablet, and mobile layouts.
FID08	Contact Validation	Client-side validation with field-level errors and success feedback.
FID09	Animations	Scroll reveal, image hover, and header behavior.
FID10	Optimized Images	High-resolution images resized/compressed for web delivery.
FID11	Favicon	Site identity displayed in browser tabs.
FID12	Static Deployment	GitHub Pages compatible release.

Table 2.2: Product Features

2.5 Operating Environment

- Development: Windows 11 and Visual Studio Code.
- Source control: Git and GitHub.
- Client: current Chrome, Edge, Firefox, Safari, or equivalent modern browser.
- Production: GitHub Pages over HTTPS.
- Build/runtime dependency: none required for normal site viewing; Node.js/Sharp is used only as a development utility for image optimization.

2.6 System Context

The visitor interacts with the static website through a browser. GitHub Pages serves HTML, CSS, JavaScript, favicon, and optimized image assets. JavaScript executes locally in the browser; there is no runtime database or application server.

Visitor / User	Tasty Recipes HTML + CSS + JS	Browser / GitHub Pages
-----------------------	--	-------------------------------

Figure 2.1: System Context Diagram

Chapter 3: REQUIREMENT SPECIFICATIONS

3.1 Introduction

This chapter defines verifiable functional, interface, and quality requirements for the implemented Version 1.0 release. Mandatory statements use SHALL.

3.2 Functional Requirements

Requirement ID	Requirement Statement
FR_01	The system SHALL provide index.html as the website entry page.
FR_02	The system SHALL provide consistent navigation to Home, Recipes, About, and Contact.
FR_03	The Recipes navigation item SHALL provide a dropdown for All Recipes and five categories.
FR_04	The system SHALL display nine recipe summaries in the catalogue.
FR_05	The system SHALL allow recipe-name search on the Recipes page.
FR_06	The system SHALL filter recipes by Breakfast, Lunch, Dinner, Healthy, and Dessert categories.
FR_07	Category query parameters SHALL preselect and apply the corresponding catalogue filter.
FR_08	Each listed recipe SHALL link to a dedicated recipe-detail page.
FR_09	Each recipe detail page SHALL display title, image, preparation metadata, ingredients, ordered instructions, and a tip.
FR_10	Ingredient items SHALL provide interactive checkboxes.
FR_11	The About page SHALL describe project purpose, story, values, and project statistics.
FR_12	The Contact page SHALL provide name, email, subject, and message fields.
FR_13	The Contact form SHALL validate required input on the client and display field-level errors.
FR_14	The system SHALL provide responsive mobile navigation.
FR_15	Content images SHALL include meaningful alternative text.
FR_16	All required local navigation links and asset paths SHALL resolve correctly.
FR_17	The system SHALL include a favicon.
FR_18	The system SHALL display a footer on all primary pages.
FR_19	The system SHALL provide visual reveal/hover interactions without blocking content access.
FR_20	The release SHALL use optimized web images while retaining clear visual quality.

Table 3.1: Functional Requirements

3.3 Fully Dressed Use Cases

3.3.1 UC-01: Browse and Filter Recipes

Item	Description
UC-ID	UC-01
Primary Actor	Visitor
Brief Description	Visitor opens the catalogue, searches, or selects a category.
Preconditions	Website is accessible in a modern browser.
Trigger	Visitor selects Recipes, a dropdown category, or enters search text.
Basic Flow	1. System opens recipes.html. 2. Catalogue displays nine recipes. 3. Visitor searches or selects a category. 4. JavaScript updates visible cards. 5. No-results feedback appears when appropriate.
Alternative Flow	A category query parameter automatically activates the matching filter.
Postcondition	A relevant set of recipe cards remains available for selection.

Table 3.2: UC-01 Browse and Filter Recipes

3.3.2 UC-02: View Recipe Details

Item	Description
UC-ID	UC-02
Primary Actor	Visitor
Brief Description	Visitor opens a recipe and reads complete information.
Preconditions	Recipe card or related-recipe link is available.
Trigger	Visitor selects View Recipe.

Basic Flow	1. Browser opens the selected detail page. 2. Page displays hero image and metadata. 3. Ingredients are displayed with checkboxes. 4. Six ordered preparation steps are shown. 5. A cooking tip and related recipes are displayed.
Postcondition	Visitor can read the recipe and navigate elsewhere.

Table 3.3: UC-02 View Recipe Details

3.3.3 UC-03: View About Information

Item	Description
UC-ID	UC-03
Primary Actor	Visitor
Brief Description	Visitor reads the website purpose and project story.
Preconditions	Website is accessible.
Trigger	Visitor selects About.
Basic Flow	System opens about.html and displays the hero, story, values, statistics, and CTA.
Postcondition	About information is visible.

Table 3.4: UC-03 View About Information

3.3.4 UC-04: Use Contact Form

Item	Description
UC-ID	UC-04
Primary Actor	Visitor
Brief Description	Visitor enters contact-form information and receives validation feedback.
Preconditions	Contact page is open and JavaScript is available.
Trigger	Visitor submits the form.
Basic Flow	1. Visitor enters name, email, subject, and message. 2. System validates minimum lengths and email format. 3. Invalid fields show errors. 4. Valid input shows a success message and resets the form.
Alternative Flow	Invalid or incomplete input remains on the page for correction.
Postcondition	No data is stored or transmitted; validation is demonstrative only.

Table 3.5: UC-04 Use Contact Form

3.4 External Interface Requirements

ID	Requirement
IR_01	Every primary page SHALL display the shared site header and navigation.
IR_02	The Recipes dropdown SHALL expose category destinations using clear labels.
IR_03	Form controls SHALL have visible labels and accessible input types.
IR_04	Recipe content SHALL visually distinguish metadata, ingredients, instructions, tips, and related recipes.
IR_05	Responsive layouts SHALL avoid horizontal overflow at common mobile widths.
IR_06	The deployed website SHALL be accessible over HTTPS.

Table 3.6: External Interface Requirements

3.5 Non-Functional Requirements

NFR ID	Category	Description
NFR01	Usability	A visitor SHOULD reach any primary section through the main navigation.
NFR02	Accessibility	Semantic landmarks, labels, logical headings, keyboard-reachable links, and meaningful alt text SHALL be used where applicable.
NFR03	Compatibility	The site SHALL render without critical content or navigation loss in current desktop and mobile browsers.
NFR04	Responsiveness	Layouts SHALL adapt to desktop, tablet, and

		mobile viewports.
NFR05	Maintainability	Files SHALL use meaningful names and organized directories; shared CSS/JS SHALL minimize duplication.
NFR06	Reliability	The release SHALL contain no known broken internal links or required local images.
NFR07	Performance	Large source images SHOULD be resized/compressed for web delivery without visible quality loss.
NFR08	Security	Passwords, tokens, private keys, node_modules, and local image backups SHALL NOT be committed.
NFR09	Privacy	Version 1.0 SHALL NOT persist or transmit contact-form data.
NFR10	Portability	The website SHALL deploy on a standard static hosting service.

Table 3.7: Non-Functional Requirements

3.6 Acceptance Criteria

- All nine recipe cards open the intended detail page.
- Recipes dropdown works on desktop and mobile.
- Category dropdown links open the catalogue with the correct filter active.
- Search and filter behavior produce correct visible cards and no-results feedback.
- All primary pages and recipe pages display the shared navigation correctly.
- Contact validation handles valid and invalid input as specified.
- Optimized images remain sharp and no required image is broken.
- No obsolete 'Recipe coming soon' text remains.
- Only intentional 'Explore Recipes' CTA text remains on Home/About.
- The site is usable at mobile, tablet, laptop, and large-desktop widths.
- GitHub Pages deployment loads over HTTPS.

Chapter 4: DESIGN SPECIFICATIONS

4.1 Introduction

The design uses a reusable front-end architecture: semantic page markup, a shared stylesheet, shared JavaScript, a centralized image library, and consistent navigation. The interface uses a warm orange accent, large food imagery, card-based content, responsive grids, and restrained animation.

4.2 System Architecture

- Presentation layer: HTML5 pages and semantic content.
- Style layer: assets/css/style.css.
- Behavior layer: assets/js/main.js.
- Media layer: assets/images/ optimized recipe and UI imagery.
- Hosting layer: GitHub Pages static hosting.
- Development utility: Node.js + Sharp for batch image optimization.

4.3 Website Structure

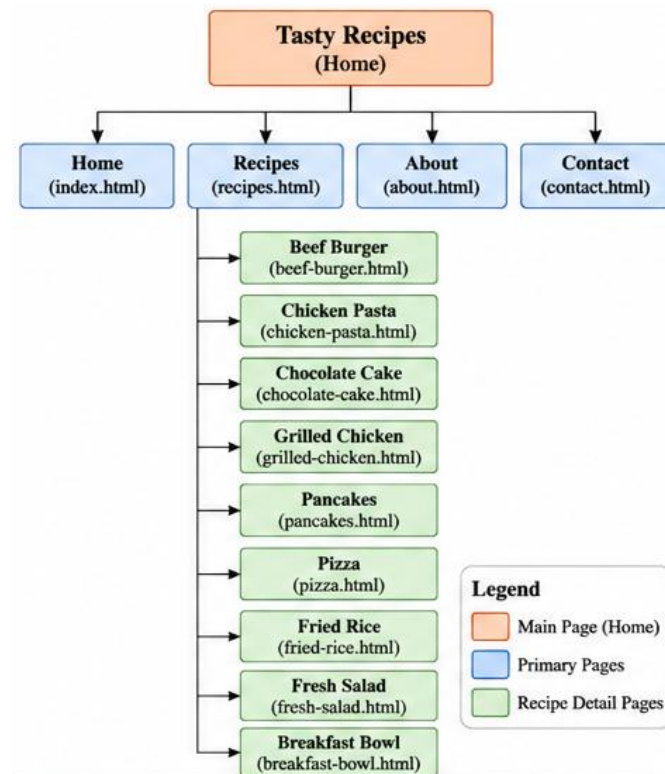


Figure 4.1: Website Navigation / Site Map

4.4 Use Case Diagram

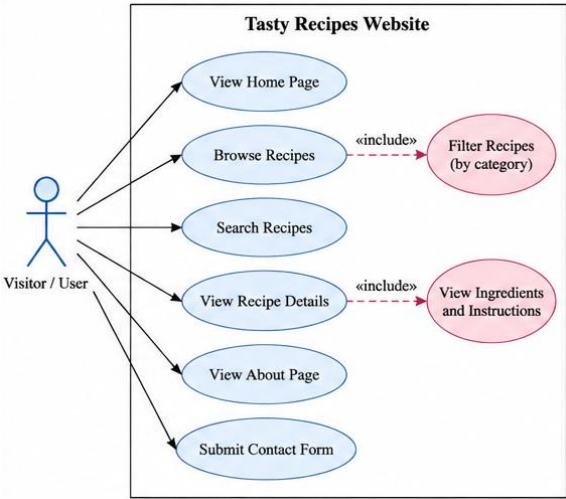


Figure 4.2: Use Case Diagram

4.5 Block Diagram

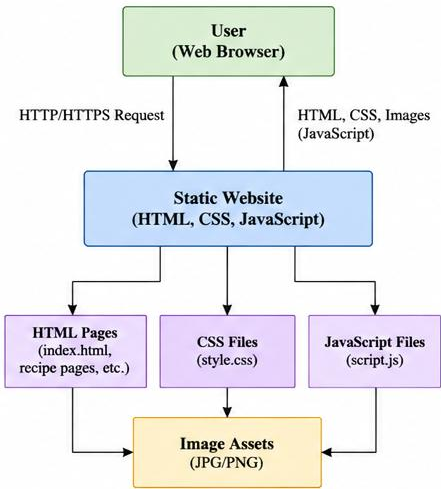


Figure 4.3: High-Level Block Diagram

4.6 Class Diagram

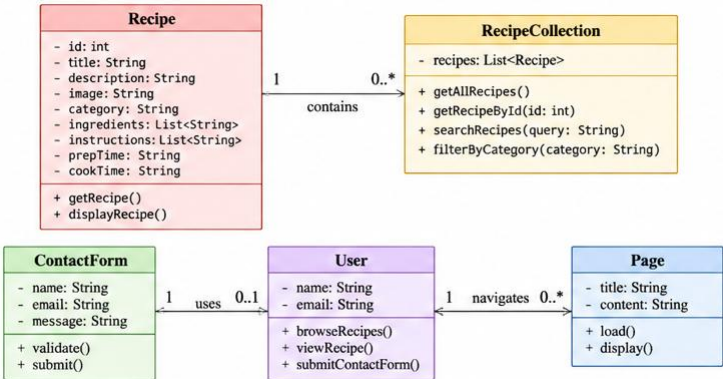


Figure 4.4: Conceptual Class Diagram

4.7 Sequence Diagram

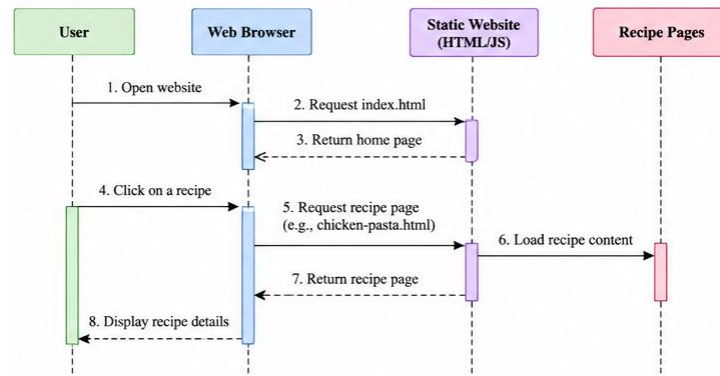


Figure 4.5: Sequence Diagram - View Recipe Details

4.8 File and Folder Design

```
01-tasty-recipes-html/  
├── index.html  
├── recipes.html  
├── about.html  
├── contact.html  
├── recipes/  
│   ├── chicken-pasta.html  
│   ├── beef-burger.html  
│   ├── pancakes.html  
│   ├── grilled-chicken.html  
│   ├── pizza.html  
│   ├── chocolate-cake.html  
│   ├── fresh-salad.html  
│   ├── fried-rice.html  
│   └── breakfast-bowl.html  
├── assets/  
│   ├── css/style.css  
│   ├── js/main.js  
│   └── images/  
├── scripts/optimize-images.js  
├── docs/Tasty_Recipes_SRS.pdf  
├── .gitignore  
├── package.json  
├── package-lock.json  
└── README.md
```

4.9 User Interface Design

The following spaces are intentionally reserved for final screenshots from the implemented project. Insert screenshots after the website is finalized so the SRS documents the actual release rather than a mock-up. Use full-width, clear captures with the browser chrome cropped where appropriate.

FRESH • SIMPLE • DELICIOUS

Discover recipes you'll love to cook.

Explore simple homemade recipes with easy-to-follow instructions, everyday ingredients and delicious results.

[Explore Recipes](#)[Featured Recipes](#)

POPULAR CHOICES

Featured Recipes

[View All Recipes →](#)

Popular



40 min 4 servings

Chicken Pasta

Creamy chicken pasta prepared with simple ingredients for a satisfying homemade meal.

[View Recipe →](#)

Classic



30 min 4 servings

Beef Burger

A juicy homemade beef burger layered with fresh vegetables, cheese and flavorful sauce.

[View Recipe →](#)

Breakfast



25 min 4 servings

Fluffy Pancakes

Light and fluffy pancakes that make a simple and delicious breakfast for any morning.

[View Recipe →](#)

Figure 4.6: Home Page UI

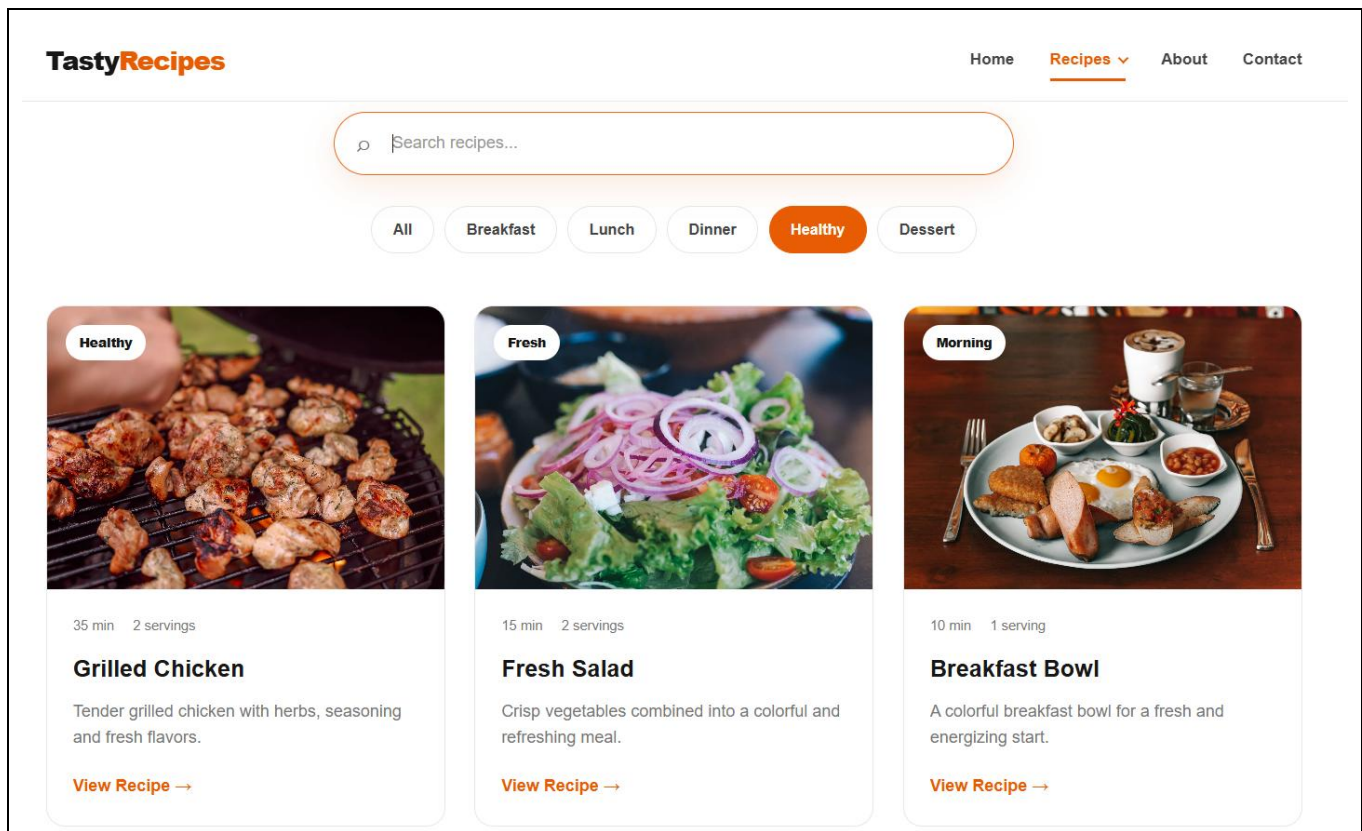


Figure 4.7: Recipe Catalogue UI

BREAKFAST • HEALTHY

Fresh Breakfast Bowl.

A colorful breakfast bowl layered with creamy yogurt, fresh fruit, oats and crunchy toppings for a quick and refreshing start to the day.



PREP TIME
10 min

COOK TIME
0 min

TOTAL TIME
10 min

SERVINGS
1

DIFFICULTY
Easy

WHAT YOU'LL NEED

Ingredients

- ☐ 1 cup plain or Greek yogurt
- ☐ ½ banana, sliced
- ☐ ½ cup fresh berries
- ☐ ¼ cup rolled oats
- ☐ 2 tablespoons granola
- ☐ 1 tablespoon chopped nuts
- ☐ 1 teaspoon chia seeds
- ☐ 1 teaspoon honey

STEP BY STEP

Instructions

- 01 Prepare the fruit**
Wash the berries thoroughly and slice the banana into even bite-sized pieces.
- 02 Add the yogurt**
Spoon the yogurt into a clean serving bowl and spread it evenly to create the base of the breakfast bowl.
- 03 Add the fruit**
Arrange the banana slices and fresh berries over the yogurt.
- 04 Add oats and granola**
Sprinkle the rolled oats and granola across the bowl for extra texture.
- 05 Add the toppings**
Finish with chopped nuts and chia seeds, distributing them evenly across the bowl.
- 06 Finish and serve**
Drizzle a small amount of honey over the top and serve immediately while the ingredients are fresh.

Figure 4.8: Recipe Detail UI

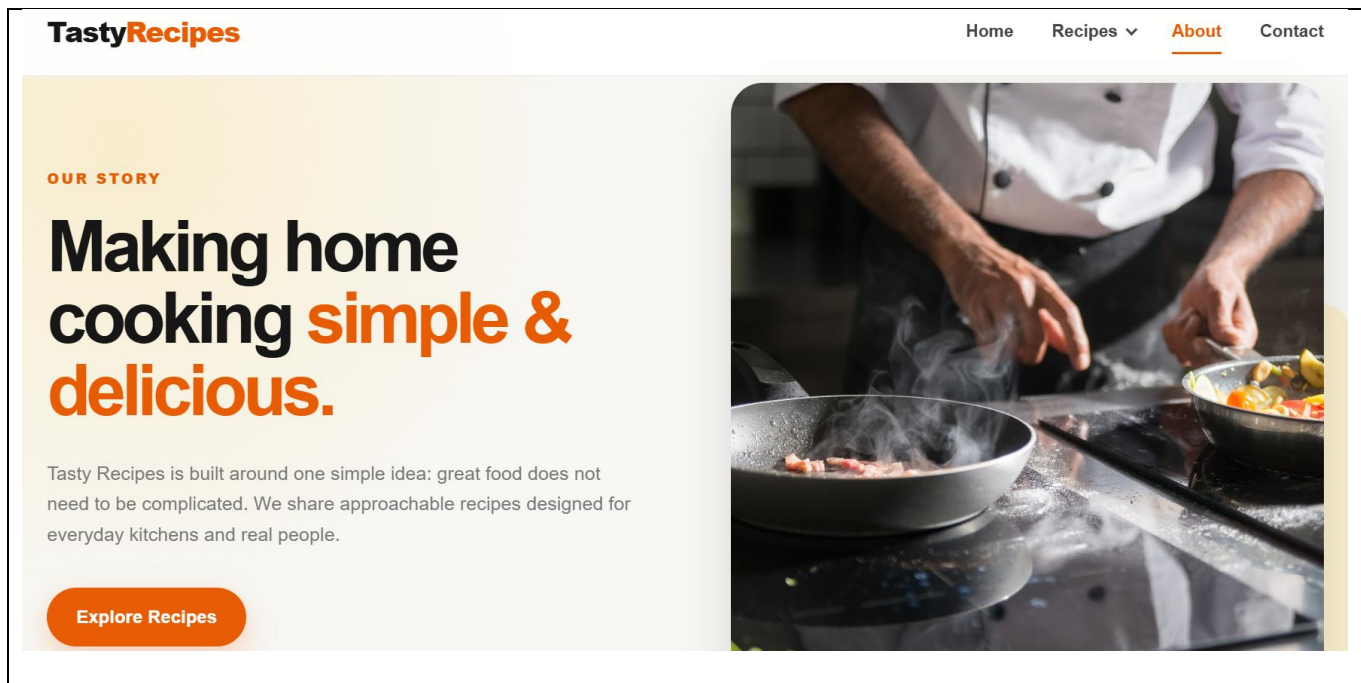


Figure 4.9: About Page UI

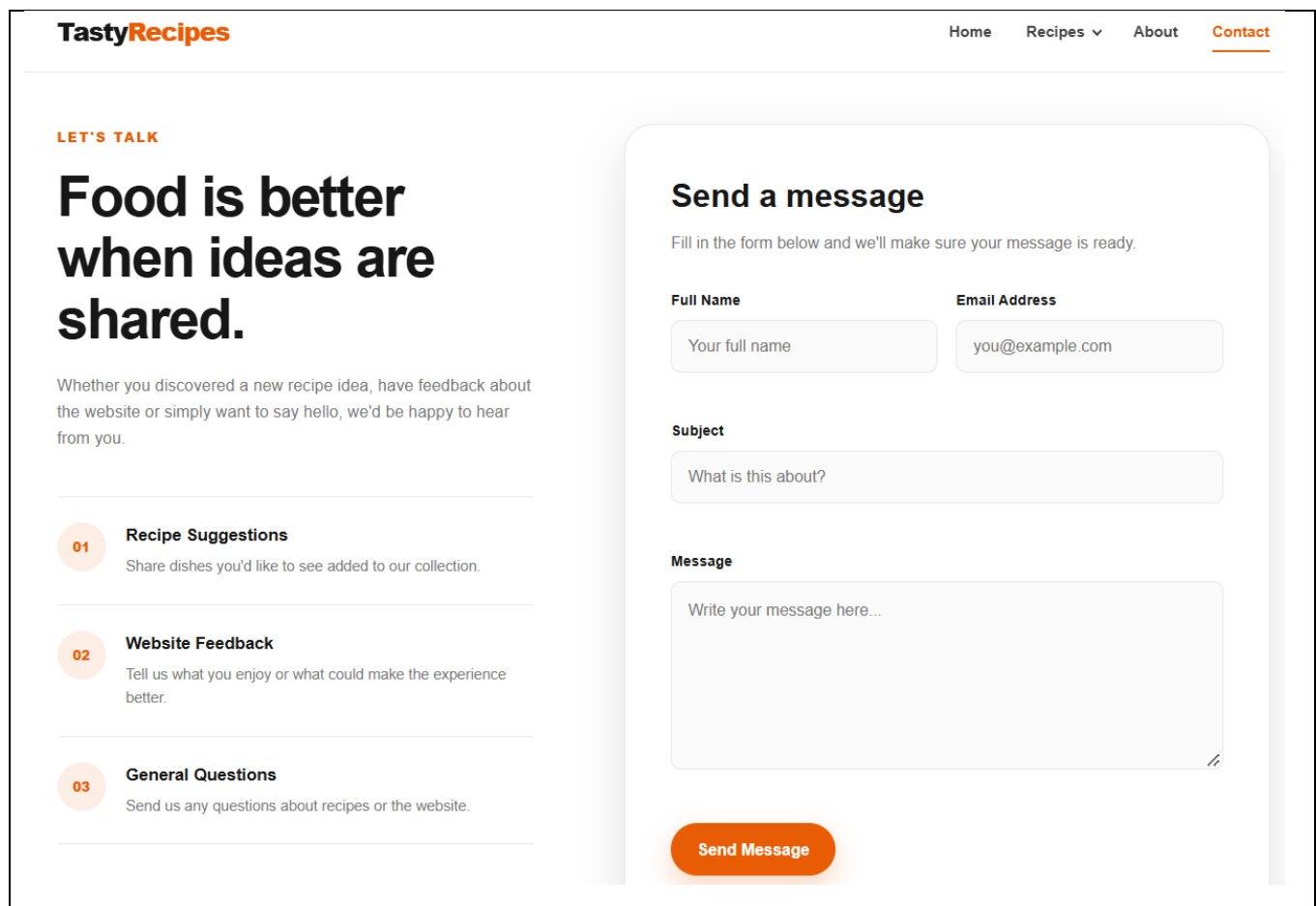


Figure 4.10: Contact Page UI

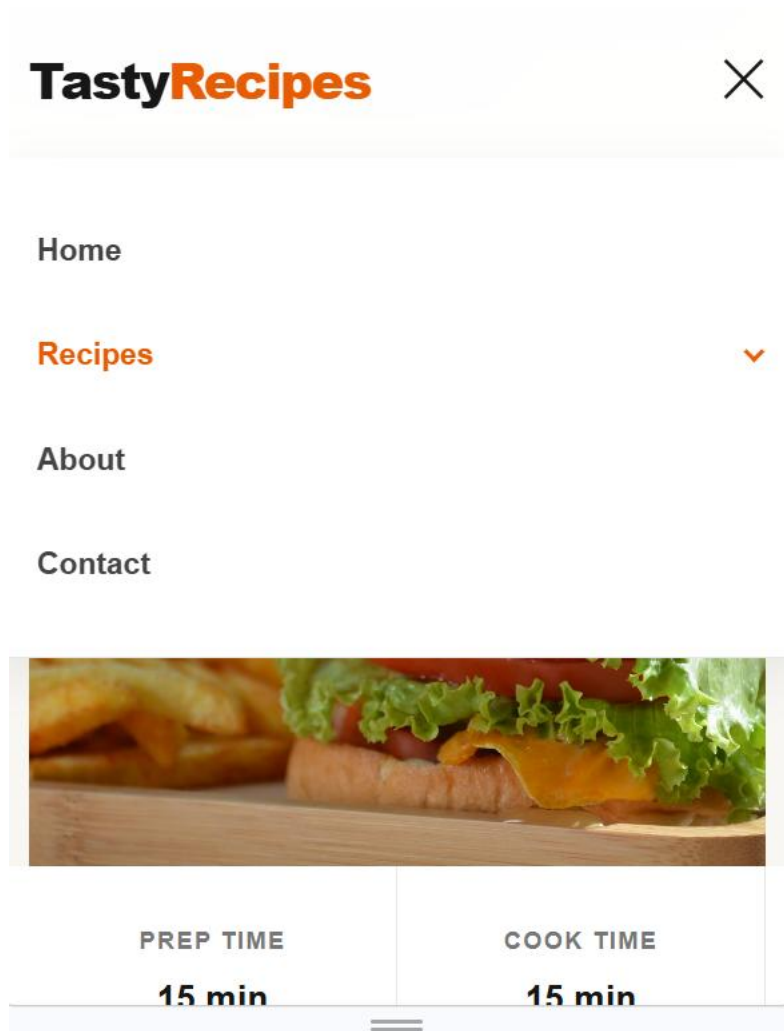


Figure 4.11: Mobile / Responsive UI

Chapter 5: TEST SPECIFICATIONS

5.1 Introduction

Testing verifies functional requirements, responsive behavior, navigation consistency, image integrity, form validation, accessibility basics, repository hygiene, and deployed behavior.

5.2 Test Strategy

- Functional testing of links, catalogue, search, filters, dropdowns, recipe pages, and form validation.
- Responsive testing at approximately 375 px mobile, tablet, laptop, and large-desktop widths.
- Compatibility checks in current Chrome and Edge.
- Accessibility review of headings, labels, alt text, keyboard navigation, and focus behavior.
- Performance review of optimized image sizes and visual sharpness.
- Deployment testing on the GitHub Pages URL.
- Repository review using git status before release.

5.3 Test Cases

Test ID	Test Description	Test Action	Expected Result	Requirement
TC_01	Home Page	Open index.html / deployed root.	Home page loads with hero, featured recipes, navigation, and footer.	FR_01
TC_02	Global Navigation	Use Home, Recipes, About, Contact on root and recipe pages.	Correct destination opens; relative paths work.	FR_02, FR_16
TC_03	Recipes Dropdown	Hover on desktop and tap on mobile.	Dropdown exposes All Recipes and five categories.	FR_03, FR_14
TC_04	Recipe Catalogue	Open recipes.html.	Nine recipe cards are displayed.	FR_04
TC_05	Search	Search for a recipe name.	Matching cards remain; unrelated cards hide.	FR_05
TC_06	Category Filter	Select each category button.	Only matching recipe cards remain visible.	FR_06
TC_07	URL Category Filter	Open recipes.html?category=dessert.	Dessert filter activates and matching recipe is shown.	FR_07
TC_08	Recipe Detail Links	Open all nine View Recipe links.	Each opens the intended complete recipe page.	FR_08, FR_09
TC_09	Ingredient Checkbox	Select ingredient checkboxes.	Checked ingredients show intended visual state.	FR_10
TC_10	Contact Validation	Submit invalid and valid sample input.	Errors appear for invalid input; valid input shows success feedback.	FR_12, FR_13
TC_11	Responsive Layout	Test mobile/tablet/desktop widths.	No critical overflow; menu and grids adapt.	FR_14, NFR04
TC_12	Image Integrity	Open catalogue and detail pages.	Required images load sharply with meaningful alt text.	FR_15, FR_20
TC_13	Obsolete Content	Search project for placeholder recipe text.	No 'Recipe coming soon' remains; only intentional CTAs remain.	NFR06
TC_14	Repository Security	Review git status and .gitignore.	No secrets, node_modules, or image backups are staged.	NFR08
TC_15	Deployment	Open live GitHub Pages site over HTTPS.	Site loads and critical navigation/interactions work.	NFR10, IR_06

Table 5.1: Test Cases

5.4 Requirements Traceability Matrix

Requirement	Design / Implementation	Verification
FR_01	index.html	TC_01
FR_02, FR_16	Shared navigation / relative paths	TC_02
FR_03, FR_14	Dropdown HTML/CSS/JS	TC_03
FR_04	recipes.html	TC_04
FR_05	main.js search logic	TC_05
FR_06	filter buttons + main.js	TC_06
FR_07	URLSearchParams category logic	TC_07
FR_08-FR_10	recipes/*.html	TC_08, TC_09
FR_12-FR_13	contact.html + main.js	TC_10
NFR04	Responsive CSS	TC_11
FR_15, FR_20	assets/images + alt attributes	TC_12
NFR06	Project-wide link/content review	TC_13
NFR08	.gitignore / repository	TC_14
NFR10 / IR_06	GitHub Pages	TC_15

Table 5.2: Requirements Traceability Matrix

5.5 Final QA Checklist

- Home hero, CTA links, featured recipe links, and footer verified.
- Recipes page search, all category filters, dropdown links, and no-results state verified.
- All nine recipe-detail pages verified for image, metadata, ingredients, instructions, tip, and related links.
- About and Contact layouts verified.
- Contact validation verified for invalid and valid input.
- Mobile navigation and dropdown verified.
- No broken images or internal links observed.
- High-resolution images optimized and visually checked after compression.
- Favicon visible on root and recipe-detail pages.
- Project-wide search confirms obsolete recipe placeholders are removed.
- Final Git status reviewed before commit and push.

Chapter 6: DEPLOYMENT AND CONCLUSION

6.1 Deployment Requirements

- Source code is maintained in the GitHub repository.
- The release is based on the main branch.
- index.html is the deployed entry point.
- All relative paths work under the repository base URL.
- GitHub Pages serves the project over HTTPS.
- README.md documents the final feature set and live deployment link.

6.2 Deployment Procedure

1. Complete implementation and final local QA.
2. Run git status and review only intended changes.
3. Stage release files with git add .
4. Commit with a professional Conventional Commit-style message.
5. Push main to GitHub.
6. Allow GitHub Pages to redeploy.
7. Open the live URL and repeat critical tests.
8. Create the v1.0.0 release tag after successful verification.

6.3 Current Limitations

- No backend or database.
- Contact form validates input but does not send or store messages.
- Recipe content is manually maintained in HTML.
- No authentication, user profiles, favorites, ratings, or comments.
- Search/filtering operates only on the static catalogue currently included in the page.

6.4 Future Enhancements

- Backend contact processing in a later full-stack project.
- Database-driven recipe management.
- Authentication and saved favorites.
- Admin recipe management.
- REST API integration.
- Automated testing and CI/CD.
- Progressive Web App capabilities where appropriate.

6.5 Conclusion

Tasty Recipes Version 1.0 demonstrates a complete front-end software-development cycle rather than an HTML-only exercise. The release combines semantic structure, responsive visual design, reusable CSS, Vanilla JavaScript interactions, optimized media, testing, version control, documentation, and static deployment. Its scope remains intentionally front-end focused so backend, database, authentication, and API capabilities can be introduced in later projects without overengineering this release.

Appendix A: GLOSSARY

Term	Definition
HTML5	Markup language used to structure web content.
CSS3	Style-sheet language used for layout, responsiveness, and presentation.
Vanilla JavaScript	JavaScript used without a front-end framework.
Static Website	Website served as prebuilt client files without server-side application logic.
Responsive Design	Layout approach that adapts to different viewport sizes.
Functional Requirement	Behavior or capability the system must provide.
Non-Functional Requirement	Quality attribute or constraint applied to the system.
Use Case	Structured description of an actor achieving a goal with the system.
Acceptance Criteria	Conditions used to determine whether implementation is acceptable.
Git	Distributed version-control system.
GitHub Pages	Static hosting service used to publish repository content.
Sharp	Node.js image-processing library used here as a development utility.

Appendix B: USER MANUAL

B.1 Opening the Website

Open the deployed GitHub Pages URL or open index.html locally in a modern browser.

B.2 Main User Actions

Action	Procedure
Browse all recipes	Select Recipes, then All Recipes.
Browse a category	Open Recipes dropdown and choose Breakfast, Lunch, Dinner, Healthy, or Dessert.
Search recipes	On the Recipes page, type part of a recipe name in the search field.
Open a recipe	Select View Recipe on any recipe card.
Check ingredients	Use the ingredient checkboxes on a recipe-detail page.
Read project information	Select About.
Validate contact input	Open Contact, complete the fields, and submit to view validation feedback.
Use on mobile	Open the hamburger menu; tap Recipes once to expand its category submenu.

Table B.1: Main User Actions

B.3 Screenshot Completion Instructions

Before final academic/portfolio submission, replace each screenshot placeholder in Section 4.9 with a clean capture from the final deployed release. Recommended captures: Home desktop, Recipes catalogue with filters visible, one recipe-detail page, About, Contact, and one mobile navigation view. Keep the figure captions and update the List of Figures page numbers if pagination changes.

BIBLIOGRAPHY

- [1] ISO/IEC/IEEE 29148:2018, Systems and software engineering - Life cycle processes - Requirements engineering.
- [2] WHATWG, HTML Living Standard.
- [3] W3C, Web Content Accessibility Guidelines (WCAG).
- [4] MDN Web Docs, HTML, CSS, and JavaScript reference documentation.
- [5] Git documentation, distributed version-control concepts.
- [6] GitHub Docs, GitHub Pages documentation.